Natural Language Processing 2

LangChain

Fatemeh Dehrouyeh



Contents

1. Introduction to LangChain
   - No API
   - OpenAI API
   - Google Gemini API
   - Groq API
2. Project Example
3. Evaluation in RAG

Current Section

1. Introduction to LangChain
   - No API
   - OpenAI API
   - Google Gemini API
   - Groq API
2. Project Example
3. Evaluation in RAG

What is LangChain?

LangChain is a framework for building applications with large language models.

It helps connect an LLM to:

- Documents
- Tools
- APIs
- Memory
- Databases
- Retrieval systems

Key idea

LangChain is not the LLM. It is the layer that connects the LLM to the rest of your system.

LangChain in a RAG Pipeline

For RAG, LangChain can help with:

- Loading documents
- Splitting text into chunks
- Creating embeddings
- Storing vectors
- Retrieving relevant chunks
- Sending context to the LLM
- Generating an answer

Documents → Chunks → Embeddings → Vector Store → Retriever → LLM

Manual RAG vs LangChain

Manual RAG code:

```
chunks = build_chunks(documents)
chunk_embeddings = create_embeddings(chunks)
results = vector_search(question, chunks, chunk_embeddings)
answer = generate_answer(question, results)
```

LangChain does the same workflow with reusable components.

How to Think About LangChain

Component	Role
OpenAI, Claude, Gemini	Generate answers
SentenceTransformer	Create embeddings
FAISS, Pinecone, Chroma	Store and search vectors
LangChain	Connect everything together

Simple definition

LangChain is an orchestration framework for LLM applications.

Current Subsection

1. Introduction to LangChain
   - No API
   - OpenAI API
   - Google Gemini API
   - Groq API
2. Project Example
3. Evaluation in RAG

Code—Imports

```
# ============================================================
# Imports
# ============================================================

#!pip -q install langchain langchain-community langchain-huggingface langchain-chroma chromadb sentence-transformers

import re

from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda, RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
```

Code—Creating Documents

```
documents = [
    Document(
        page_content="""
        Customers can request a refund within 30 days of purchase.
        The item must be unused and returned in its original packaging.
        Refunds are issued to the original payment method.
        """,
        metadata={"doc_id": "doc1", "title": "Refund Policy"}
    ),
    Document(
        page_content="""
        Damaged products must be reported within 7 days of delivery.
        Customers should send a photo of the damaged item.
        After approval, the company will offer a replacement or refund.
        """,
        metadata={"doc_id": "doc2", "title": "Damaged Products"}
    ),
    Document(
        page_content="""
        Standard shipping takes 3 to 5 business days.
        Express shipping takes 1 to 2 business days.
        International shipping may take longer.
        """,
        metadata={"doc_id": "doc3", "title": "Shipping Policy"}
    ),
    Document(
        page_content="""
        Section 12.4 states that either party may terminate the agreement
        with 30 days written notice if the other party violates the contract.
        """,
        metadata={"doc_id": "doc4", "title": "Section 12.4 Termination"}
    ),
    Document(
        page_content="""
        A physician must review the patient record before treatment.
        The doctor should confirm allergies and medication history.
        """,
        metadata={"doc_id": "doc5", "title": "Medical Synonyms Example"}
    )
]
```

Code—Chunking

```python
# ============================================================
# Split documents into chunks
# ============================================================

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=180,
    chunk_overlap=40
)

chunks = text_splitter.split_documents(documents)

print(f"Number of original documents: {len(documents)}")
print(f"Number of chunks: {len(chunks)}")

for i, chunk in enumerate(chunks):
    chunk.metadata["chunk_id"] = f"{chunk.metadata['doc_id']}_chunk_{i}"

    print("\n" + "-" * 60)
    print("Chunk ID:", chunk.metadata["chunk_id"])
    print("Title:", chunk.metadata["title"])
    print("Text:", chunk.page_content.strip())
```

RecursiveCharacterTextSplitter

`RecursiveCharacterTextSplitter` is a LangChain tool for splitting long documents into smaller chunks before creating embeddings.

It tries to split text in a natural order:

- Paragraphs
- Lines
- Spaces
- Characters

Why it matters in RAG

Smaller overlapping chunks help the retriever find the most relevant parts of a document instead of searching the full document at once.

Code—Create embeddings

This line loads a Hugging Face sentence embedding model so LangChain can convert each text chunk and user question into vectors for semantic search.

```
# ============================================================
# Create embeddings
# ============================================================

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

print("Embedding model loaded.")
```

Code—Storing Chunks

This code builds a Chroma vector database from the chunks so the retriever can later search for chunks with meanings similar to the user's question.

```
# ============================================================
# Store chunks in Chroma vector database
# ============================================================

vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embedding_model,
    collection_name="teaching_rag_demo"
)

print("Chunks stored in vector database.")
```

```
Chunks stored in vector database.
```

Vector Databases in RAG

A vector database stores text chunks as embedding vectors and searches for chunks with similar meaning to the user question.

- **Chroma:** an open-source vector database used to store embeddings, metadata, and documents for similarity search.
- **FAISS:** an open-source library from Meta for efficient similarity search over dense vectors.
- **Pinecone:** a managed vector database service that stores, indexes, and searches embeddings through a cloud API.

In RAG

Chroma, FAISS, and Pinecone all help retrieve the most relevant chunks before sending context to the LLM.

Code—Create retriever

This part searches the vector database for the 3 chunks most semantically similar to the question and prints their text and metadata.

```
# ============================================================
# Create retriever
# ============================================================

retriever = vector_store.as_retriever(
    search_kwargs={"k": 3}
)

question = "What is the return deadline for damaged products?"

retrieved_docs = retriever.invoke(question)

print("Question:", question)

for doc in retrieved_docs:
    print("\n" + "-" * 60)
    print("Title:", doc.metadata["title"])
    print("Chunk ID:", doc.metadata["chunk_id"])
    print("Text:", doc.page_content.strip())
```

Code—Create retriever Result

```
Question: What is the return deadline for damaged products?

------------------------------------------------------------
Title: Damaged Products
Chunk ID: doc2_chunk_2
Text: Damaged products must be reported within 7 days of delivery.
        Customers should send a photo of the damaged item.

------------------------------------------------------------
Title: Refund Policy
Chunk ID: doc1_chunk_0
Text: Customers can request a refund within 30 days of purchase.
        The item must be unused and returned in its original packaging.

------------------------------------------------------------
Title: Damaged Products
Chunk ID: doc2_chunk_3
Text: After approval, the company will offer a replacement or refund.
```

Code—Helper functions for RAG

```python
# ============================================================
# Helper functions for RAG
# ============================================================

def format_docs(docs):
    """
    Turns retrieved documents into one context string.
    """
    formatted = []

    for doc in docs:
        title = doc.metadata.get("title", "Unknown Title")
        chunk_id = doc.metadata.get("chunk_id", "Unknown Chunk")

        formatted.append(
            f"Title: {title}\n"
            f"Chunk ID: {chunk_id}\n"
            f"Text: {doc.page_content.strip()}"
        )

    return "\n\n".join(formatted)
```

This function formats the retrieved documents (chunks) into one readable context block that can be sent to the LLM.

Helper functions for RAG—Continued

```python
def simple_extract_answer(inputs):
    """
    Simple generator.

    This is not a real LLM.
    It selects the sentence from the retrieved context
    with the highest word overlap with the question.
    """

    question = inputs["question"]
    context = inputs["context"]

    sentences = re.split(r"(?<=[.!?])\s+", context)

    question_words = set(re.findall(r"\b\w+\b", question.lower()))

    best_sentence = None
    best_score = 0

    for sentence in sentences:
        sentence_words = set(re.findall(r"\b\w+\b", sentence.lower()))
        score = len(question_words.intersection(sentence_words))

        if score > best_score:
            best_score = score
            best_sentence = sentence

    if best_sentence:
        return best_sentence.strip()

    return "I do not know based on the provided context."
```

Code—Create a LangChain RAG chain

```
# ============================================================
# Create a LangChain RAG chain
# ============================================================

rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough()
    }
    | RunnableLambda(simple_extract_answer)
)

question = "What is the return deadline for damaged products?"

answer = rag_chain.invoke(question)

print("Question:")
print(question)

print("\nAnswer:")
print(answer)
```

```
Question:
What is the return deadline for damaged products?

Answer:
Title: Damaged Products
Chunk ID: doc2_chunk_3
Text: After approval, the company will offer a replacement or refund.
```

LangChain Chain Structure

LangChain uses the pipe operator | to connect steps in a pipeline.

Meaning of |

The output from the left side becomes the input to the right side.

$$\text{Input} \rightarrow \text{Step 1} \rightarrow \text{Step 2} \rightarrow \text{Output}$$

RAG Chain Example

```
rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough()
    }
    | RunnableLambda(simple_extract_answer)
)
```

This chain creates two inputs:

- `context`: retrieved chunks formatted into one context string
- `question`: the original question passed forward unchanged

Then `RunnableLambda(simple_extract_answer)` receives both inputs and generates the final answer.

What Happens Inside the Chain?

When we run:

```
rag_chain.invoke(question)
```

LangChain sends the same question into two paths.

- **Context path:** question → retriever → chunks → formatted context
- **Question path:** question → passed forward unchanged

Then both are combined into one dictionary and passed to `simple_extract_answer`.

Final Answer Step

The last part of the chain is:

```
| RunnableLambda(simple_extract_answer)
```

Its job is to take the dictionary:

```
{"context", "question"}
```

and use it to produce the final answer.

In this demo

`simple_extract_answer` is not a real LLM. It selects the sentence from the retrieved context with the highest word overlap with the question.

Equivalent Plain Python

The LangChain chain is similar to writing this manually:

```
question = "What is the return deadline?"

docs = retriever.invoke(question)
context = format_docs(docs)

inputs = {
    "context": context,
    "question": question
}

answer = simple_extract_answer(inputs)
```

Key idea

LangChain lets us write retrieval, formatting, and answer generation as one reusable pipeline.

RunnableLambda and RunnablePassthrough

In LangChain, a `Runnable` is one step in a chain.

- `RunnableLambda:` wraps a normal Python function so it can be used inside a LangChain pipeline.
- `RunnablePassthrough:` passes the input forward unchanged.

In our RAG chain

`RunnableLambda(format_docs)` formats retrieved chunks into context, while `RunnablePassthrough()` keeps the original question unchanged.

Code—Answer

```
# ============================================================
# Show answer with sources
# ============================================================
def rag_with_sources(question):
    """
    Runs retrieval and answer generation.
    Returns both the answer and the retrieved sources.
    """

    docs = retriever.invoke(question)
    context = format_docs(docs)

    answer = simple_extract_answer({"question": question, "context": context})

    return {
        "question": question,
        "answer": answer,
        "sources": docs}

result = rag_with_sources("What is the return deadline for damaged products?")

print("Question:")
print(result["question"])

print("\nAnswer:")
print(result["answer"])

print("\nSources:")
for doc in result["sources"]:
    print("\n" + "-" * 60)
    print("Title:", doc.metadata["title"])
    print("Chunk ID:", doc.metadata["chunk_id"])
    print("Text:", doc.page_content.strip())
```

Code—Result

```
Question:
What is the return deadline for damaged products?

Answer:
Title: Damaged Products
Chunk ID: doc2_chunk_3
Text: After approval, the company will offer a replacement or refund.

Sources:

------------------------------------------------------------
Title: Damaged Products
Chunk ID: doc2_chunk_2
Text: Damaged products must be reported within 7 days of delivery.
        Customers should send a photo of the damaged item.

------------------------------------------------------------
Title: Refund Policy
Chunk ID: doc1_chunk_0
Text: Customers can request a refund within 30 days of purchase.
        The item must be unused and returned in its original packaging.

------------------------------------------------------------
Title: Damaged Products
Chunk ID: doc2_chunk_3
Text: After approval, the company will offer a replacement or refund.
```

Code—Different Questions

```
# ============================================================
# Try different questions
# ============================================================

questions = [
    "What is the return deadline for damaged products?",
    "How long does standard shipping take?",
    "What does Section 12.4 say about termination?",
    "Who should review the patient record?",
    "What is the refund policy?"
]

for q in questions:
    result = rag_with_sources(q)

    print("\n" + "=" * 80)
    print("Question:", result["question"])
    print("Answer:", result["answer"])

    print("\nTop source:")
    top_source = result["sources"][0]
    print("Title:", top_source.metadata["title"])
    print("Chunk ID:", top_source.metadata["chunk_id"])
```

Code—Similarity Score

```
# ============================================================
# Inspect similarity scores
# ============================================================

question = "What does Section 12.4 say about termination?"

docs_and_scores = vector_store.similarity_search_with_score(
    question,
    k=3
)

print("Question:", question)

for doc, score in docs_and_scores:
    print("\n" + "-" * 60)
    print("Score:", score)
    print("Title:", doc.metadata["title"])
    print("Chunk ID:", doc.metadata["chunk_id"])
    print("Text:", doc.page_content.strip())
```

Code—Result

```
Question: What does Section 12.4 say about termination?

------------------------------------------------------------
Score: 0.7961776852607727
Title: Section 12.4 Termination
Chunk ID: doc4_chunk_5
Text: Section 12.4 states that either party may terminate the agreement
        with 30 days written notice if the other party violates the contract.

------------------------------------------------------------
Score: 1.4495662450790405
Title: Damaged Products
Chunk ID: doc2_chunk_3
Text: After approval, the company will offer a replacement or refund.

------------------------------------------------------------
Score: 1.5487844944000244
Title: Refund Policy
Chunk ID: doc1_chunk_1
Text: Refunds are issued to the original payment method.
```

Current Subsection

1. Introduction to LangChain
   - No API
   - OpenAI API
   - Google Gemini API
   - Groq API
2. Project Example
3. Evaluation in RAG

OpenAI API

Run this if you want true generation instead of simple extraction:

```
# ========================================================
# Use a real LLM if you have an OpenAI API key
# ========================================================
!pip -q install langchain-openai
import os
from getpass import getpass
from langchain_openai import ChatOpenAI

os.environ["OPENAI_API_KEY"] = getpass("Enter your OpenAI API key: ")

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

prompt = ChatPromptTemplate.from_template("""
Use the context to answer the question.
If the answer is not in the context, say "I do not know."

Question:
{question}

Context:
{context}
""")

llm_rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)

question = "What is the return deadline for damaged products?"
answer = llm_rag_chain.invoke(question)
print(answer)
```

How to find the model name?

To find the correct model name (here: gpt-4o-mini), go to the API documentation page of each provider. For OpenAI models, use:

https://developers.openai.com/api/docs/models

But since this API requires paid access, we switch to free or free-tier options such as Gemini or Groq.

Note: OPENAI_API_KEY is the standard environment variable name that LangChain's ChatOpenAI uses to find your OpenAI API key securely.

StrOutputParser

`StrOutputParser` converts the LLM output into a plain Python string.

Without it, the model may return a structured message object:

```
AIMessage(content="...")
```

With `StrOutputParser`, we get only the answer text:

```
"Damaged products must be reported within 7 days."
```

In a LangChain chain

`prompt` → `llm` → `StrOutputParser()`

Current Subsection

1. Introduction to LangChain
   - No API
   - OpenAI API
   - Google Gemini API
   - Groq API
2. Project Example
3. Evaluation in RAG

Google Gemini API key

```
# ============================================================
# Use Gemini API as the LLM
# ============================================================

!pip -q install langchain-google-genai

import os
from getpass import getpass

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

os.environ["GOOGLE_API_KEY"] = getpass("Enter your Google Gemini API key: ")

llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0
)

prompt = ChatPromptTemplate.from_template("""
Use the context to answer the question.
If the answer is not in the context, say "I do not know."

Question:
{question}

Context:
{context}
""")
```

Google Gemini API key—Continued

```
llm_rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)

question = "What is the return deadline for damaged products?"
answer = llm_rag_chain.invoke(question)

print(answer)
```

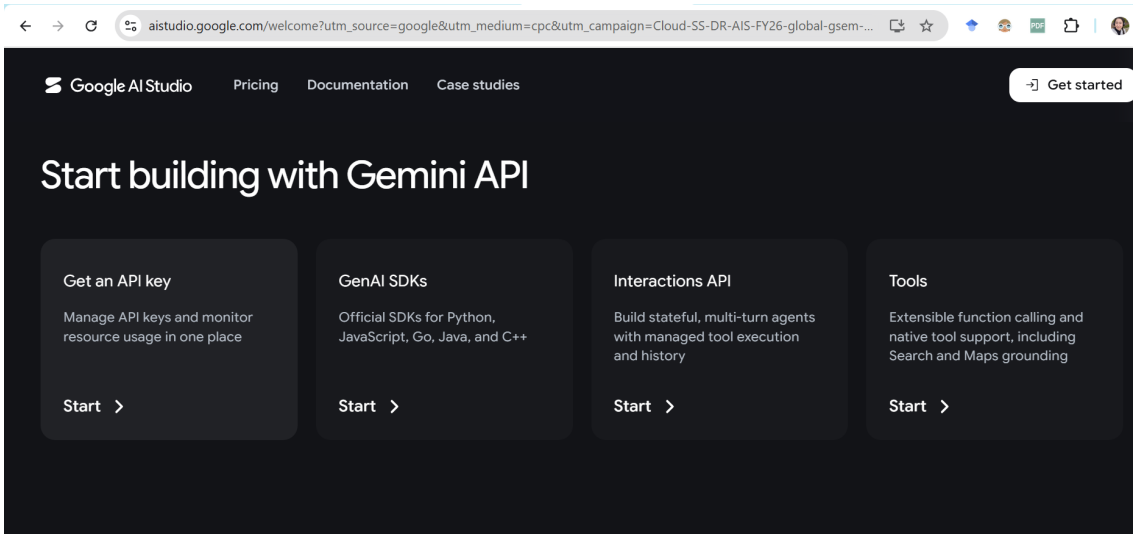
```
Enter your Google Gemini API key: ··········
I do not know.
```

How to Get a Google Gemini API Key

Go to Google AI Studio:

`https://aistudio.google.com/`

Click on **Get an API key**, then **Create API key**.



Copy the generated key and keep it private.

Gemeni Answer

Gemini answered “I do not know” because the prompt was very strict, and the question used the phrase “return deadline” while the retrieved context used different wording: “reported within 7 days.”. Even though the retriever finds the correct context:

```
question = "What is the return deadline for damaged products?"

docs = retriever.invoke(question)
context = format_docs(docs)

print("Retrieved Context:")
print(context)
```

```
Retrieved Context:
Title: Damaged Products
Chunk ID: doc2_chunk_2
Text: Damaged products must be reported within 7 days of delivery.
        Customers should send a photo of the damaged item.

Title: Refund Policy
Chunk ID: doc1_chunk_0
Text: Customers can request a refund within 30 days of purchase.
        The item must be unused and returned in its original packaging.

Title: Damaged Products
Chunk ID: doc2_chunk_3
Text: After approval, the company will offer a replacement or refund.
```

Current Subsection

1. Introduction to LangChain
   - No API
   - OpenAI API
   - Google Gemini API
   - Groq API
2. Project Example
3. Evaluation in RAG

Groq API

```
# ============================================================
# Use Groq API as the LLM
# ============================================================

!pip -q install langchain-groq

import os
from getpass import getpass

from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

os.environ["GROQ_API_KEY"] = getpass("Enter your Groq API key: ")

llm = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0
)

prompt = ChatPromptTemplate.from_template("""
Use the context to answer the question.
If the answer is not in the context, say "I do not know."

Question:
{question}

Context:
{context}
""")

llm_rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)

question = "What is the return deadline for damaged products?"
answer = llm_rag_chain.invoke(question)

print(answer)
```

How to Get a Groq API Key

Go to the Groq Console API Keys page:

`https://console.groq.com/keys`

Sign in, click **Create API Key**, give the key a name, and copy the generated key.

Important

Keep the API key private and do not paste it directly into shared notebooks or slides.

Groq Answer

Groq gives a better answer because it explains that the context contains a 7-day reporting deadline for damaged products and a 30-day general refund deadline, but does not explicitly state a separate return deadline for damaged products.

```
•••     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 137.5/137.5 kB 13.4 MB/s eta 0:00:00
    Enter your Groq API key: ··········
    I do not know.

    The context mentions the deadline for reporting damaged products (7 days) and the general refund po
```

Current Section

1 Introduction to LangChain

2 Project Example

3 Evaluation in RAG

Project Example

Real RAG Example with LangChain

This example builds a real RAG system that answers questions using content loaded from Wikipedia.

The pipeline is:

Wikipedia Article → Chunks → Embeddings → Chroma → Retriever → Groq LLM → Answer

Goal

Instead of using synthetic documents, this example retrieves information from a real article and uses it as context for answer generation.

Code—Imports

```
# ============================================================
# Install packages
# ============================================================

!pip -q install langchain langchain-community langchain-huggingface langchain-chroma langchain-groq chromadb sentence-transformers wikipedia
```

```
  Preparing metadata (setup.py) ... done
  Building wheel for wikipedia (setup.py) ... done
```

```
# ============================================================
# Imports
# ============================================================

import os
from getpass import getpass

from langchain_community.document_loaders import WikipediaLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_groq import ChatGroq

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda, RunnablePassthrough
```

Code—Adding API

```
# ========================================================
# Add Groq API key
# ========================================================

os.environ["GROQ_API_KEY"] = getpass("Enter your Groq API key: ")
```

```
Enter your Groq API key: ··········
```

Code—Load Documents

```python
# ============================================================
# Load real documents from Wikipedia
# ============================================================
# This is the "real data" part.
# You can change the query to another topic.

loader = WikipediaLoader(
    query="Retrieval-augmented generation",
    load_max_docs=2
)

documents = loader.load()

print(f"Loaded {len(documents)} documents")

for i, doc in enumerate(documents):
    print("\n" + "-" * 70)
    print("Document:", i + 1)
    print("Source:", doc.metadata.get("source"))
    print("Title:", doc.metadata.get("title"))
    print(doc.page_content[:500])
```

```
Loaded 2 documents

----------------------------------------------------------------------
Document: 1
Source: https://en.wikipedia.org/wiki/Retrieval-augmented_generation
Title: Retrieval-augmented generation
Retrieval-augmented generation (RAG) is a technique that enables large language models (LLMs) to retrieve and incorporate new inform

----------------------------------------------------------------------
Document: 2
Source: https://en.wikipedia.org/wiki/Vector_database
Title: Vector database
A vector database, vector store or vector search engine is a database that stores and retrieves embeddings of data in vector space.
```

Code—Chunking

```
# ============================================================
# Split documents into chunks
# ============================================================

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=700,
    chunk_overlap=120
)

chunks = text_splitter.split_documents(documents)

for i, chunk in enumerate(chunks):
    chunk.metadata["chunk_id"] = f"chunk_{i}"

print(f"Number of chunks: {len(chunks)}")

print("\nExample chunk:")
print(chunks[0].page_content[:700])
print("\nMetadata:")
print(chunks[0].metadata)
```

```
Number of chunks: 17

Example chunk:
Retrieval-augmented generation (RAG) is a technique that enables large language models (LLMs) to retrieve and

Metadata:
{'title': 'Retrieval-augmented generation', 'summary': 'Retrieval-augmented generation (RAG) is a technique th
```

Code—Embeddings

```
# ============================================================
# Create embeddings
# ============================================================

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

print("Embedding model loaded.")
```

```
Loading weights: 100% 103/103 [00:00<00:00, 2714.90it/s]
Embedding model loaded.
```

Code—Vector Database

```
# ============================================================
# Store chunks in Chroma vector database
# ============================================================

vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embedding_model,
    collection_name="real_wikipedia_rag_demo"
)

print("Chunks stored in Chroma.")
```

```
Chunks stored in Chroma.
```

Code—Create Retriever

```python
# ============================================================
# Create retriever
# ============================================================

retriever = vector_store.as_retriever(
    search_kwargs={"k": 4}
)

question = "What is retrieval augmented generation?"

retrieved_docs = retriever.invoke(question)

print("Question:", question)

for doc in retrieved_docs:
    print("\n" + "-" * 70)
    print("Title:", doc.metadata.get("title"))
    print("Chunk ID:", doc.metadata.get("chunk_id"))
    print("Source:", doc.metadata.get("source"))
    print(doc.page_content[:600])
```

Retriever—Result

```
Question: What is retrieval augmented generation?

----------------------------------------------------------------------
Title: Retrieval-augmented generation
Chunk ID: chunk_7
Source: https://en.wikipedia.org/wiki/Retrieval-augmented_generation
== Process ==
Retrieval-augmented generation (RAG) enhances large language models (LLMs) by incorporating an information-retrieval mechanism that allows mo

----------------------------------------------------------------------
Title: Retrieval-augmented generation
Chunk ID: chunk_0
Source: https://en.wikipedia.org/wiki/Retrieval-augmented_generation
Retrieval-augmented generation (RAG) is a technique that enables large language models (LLMs) to retrieve and incorporate new information fro

----------------------------------------------------------------------
Title: Retrieval-augmented generation
Chunk ID: chunk_2
Source: https://en.wikipedia.org/wiki/Retrieval-augmented_generation
RAG also reduces the need to retrain LLMs with new data, saving on computational and financial costs. Beyond efficiency gains, RAG also allow
The term retrieval-augmented generation (RAG) was introduced in a 2020 paper that described combining a parametric language model with a non-

----------------------------------------------------------------------
Title: Vector database
Chunk ID: chunk_14
Source: https://en.wikipedia.org/wiki/Vector_database
An especially common use-case for vector databases is in retrieval-augmented generation (RAG), a method to improve domain-specific responses
```

Code—Formatting Chunks

```python
# ============================================================
# Format retrieved chunks as context
# ============================================================

def format_docs(docs):
    formatted_docs = []

    for doc in docs:
        title = doc.metadata.get("title", "Unknown Title")
        source = doc.metadata.get("source", "Unknown Source")
        chunk_id = doc.metadata.get("chunk_id", "Unknown Chunk")

        formatted_docs.append(
            f"Title: {title}\n"
            f"Source: {source}\n"
            f"Chunk ID: {chunk_id}\n"
            f"Text: {doc.page_content}"
        )

    return "\n\n".join(formatted_docs)
```

Code—LLM and Prompt

```python
# ============================================================
# Create Groq LLM
# ============================================================

llm = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0
)
```

```python
# ============================================================
# Create RAG prompt
# ============================================================

prompt = ChatPromptTemplate.from_template("""
You are a careful RAG assistant.

Answer the question using only the context below.
If the answer is not in the context, say "I do not know."

Question:
{question}

Context:
{context}

Answer:
""")
```

Code—LangChain RAG Chain

```
# ============================================================
# Build the LangChain RAG chain
# ============================================================

rag_chain = (
    {
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)
```

Code—Question

```
# ============================================================
# Ask a question
# ============================================================

question = "What is retrieval augmented generation?"

answer = rag_chain.invoke(question)

print("Question:")
print(question)

print("\nAnswer:")
print(answer)
```

```
Question:
What is retrieval augmented generation?

Answer:
Retrieval augmented generation (RAG) enhances large language models (LLMs) by incorporating an
```

Code—More Questions

```python
# ============================================================
# Ask more real questions
# ============================================================

questions = [
    "Why is retrieval augmented generation useful?",
    "What problem does RAG try to solve?",
    "How does RAG combine retrieval and generation?",
    "What are possible limitations of retrieval augmented generation?"
]

for q in questions:
    print("\n" + "=" * 90)
    print("Question:", q)
    print("\nAnswer:")
    print(rag_chain.invoke(q))
```

```
==========================================================================================
Question: Why is retrieval augmented generation useful?

Answer:
Retrieval augmented generation is useful because it reduces the need to retrain large language models with new data, saving on computational and financial

==========================================================================================
Question: What problem does RAG try to solve?

Answer:
RAG tries to solve the problem of AI hallucinations, which have caused chatbots to describe policies that don't exist, or recommend nonexistent legal case

==========================================================================================
Question: How does RAG combine retrieval and generation?

Answer:
RAG combines retrieval and generation by first retrieving relevant information from external data sources, and then using this information along with the

==========================================================================================
Question: What are possible limitations of retrieval augmented generation?

Answer:
I do not know.
```

Code—Answer with Sources

```python
# ============================================================
# Answer with sources
# ============================================================

def answer_with_sources(question):
    docs = retriever.invoke(question)
    context = format_docs(docs)

    answer = rag_chain.invoke(question)

    print("Question:")
    print(question)

    print("\nAnswer:")
    print(answer)

    print("\nRetrieved Sources:")
    for doc in docs:
        print("\n" + "-" * 70)
        print("Title:", doc.metadata.get("title"))
        print("Chunk ID:", doc.metadata.get("chunk_id"))
        print("Source:", doc.metadata.get("source"))
        print("Text Preview:")
        print(doc.page_content[:500])


answer_with_sources("Why is retrieval augmented generation useful?")
```

Answer with Sources—Result

```
Question:
Why is retrieval augmented generation useful?

Answer:
Retrieval augmented generation is useful because it reduces the need to retrain large language models with new data, saving on computational and financ

Retrieved Sources:

--------------------------------------------------------------------------------
Title: Retrieval-augmented generation
Chunk ID: chunk_7
Source: https://en.wikipedia.org/wiki/Retrieval-augmented_generation
Text Preview:
== Process ==
Retrieval-augmented generation (RAG) enhances large language models (LLMs) by incorporating an information-retrieval mechanism that allows models to ac

--------------------------------------------------------------------------------
Title: Retrieval-augmented generation
Chunk ID: chunk_2
Source: https://en.wikipedia.org/wiki/Retrieval-augmented_generation
Text Preview:
RAG also reduces the need to retrain LLMs with new data, saving on computational and financial costs. Beyond efficiency gains, RAG also allows LLMs to
The term retrieval-augmented generation (RAG) was introduced in a 2020 paper that described combining a parametric language model with a non-parametric

--------------------------------------------------------------------------------
Title: Retrieval-augmented generation
Chunk ID: chunk_0
Source: https://en.wikipedia.org/wiki/Retrieval-augmented_generation
Text Preview:
Retrieval-augmented generation (RAG) is a technique that enables large language models (LLMs) to retrieve and incorporate new information from external

--------------------------------------------------------------------------------
Title: Vector database
Chunk ID: chunk_14
Source: https://en.wikipedia.org/wiki/Vector_database
Text Preview:
An especially common use-case for vector databases is in retrieval-augmented generation (RAG), a method to improve domain-specific responses of large l
```

Changing the RAG Topic

To create a new real RAG project, change only the Wikipedia query:

```
query = "Retrieval-augmented generation"
```

Examples:

```
query = "Natural language processing"
```

```
query = "Large language model"
```

Other Data Sources for RAG

RAG systems can retrieve information from many sources besides Wikipedia.

Source	**LangChain Loader**
Wikipedia	`WikipediaLoader`
PDF files	`PyPDFLoader`
Web pages	`WebBaseLoader`
Text files	`TextLoader`
CSV files	`CSVLoader`
Word documents	`Docx2txtLoader`
Research papers (arXiv)	`ArxivLoader`

Key idea

LangChain provides document loaders that make it easy to build RAG systems from different data sources.

Current Section

1 Introduction to LangChain

2 Project Example

3 Evaluation in RAG

Why Evaluate RAG Systems?

A RAG system has two main parts: retrieval and generation.

- The retriever finds relevant documents or chunks.
- The generator uses those chunks to produce an answer.
- Evaluation checks whether both parts work correctly.

Main idea

A RAG answer can fail because the retriever found the wrong context, or because the generator used the context incorrectly.

RAG Evaluation Pipeline



- Retriever evaluation checks whether the correct evidence was found.
- Generator evaluation checks whether the final answer is correct and grounded.

Ground Truth in RAG Evaluation

Many RAG evaluation metrics require **ground truth**.

Ground truth

Ground truth is the correct reference information used to judge the system output.

- For retriever evaluation, ground truth is usually the correct document or chunk.
- For generator evaluation, ground truth is usually the expected answer or grading rubric.
- Without ground truth, we can still use LLM-based evaluation, but the scores are less objective.

Example:

Question	What is the refund period?
Correct chunk	chunk_17
Expected answer	Customers can request a refund within 30 days.

Retriever Evaluation Metrics

Retriever metrics measure whether the correct document or chunk appears near the top of the search results.

Metric	What it measures
MRR (Mean Reciprocal Rank)	How high the first correct result appears in the ranked list.
Mean Rank	The average rank position of the correct result across all test queries.
Median Rank	The middle rank position of the correct result across all test queries.
Hit@K	Whether the correct result appears in the top K results.
Recall@K	Whether the relevant evidence appears within the top K retrieved results.
Precision@K	What fraction of the top K retrieved results are relevant.
nDCG@K (Normalized Discounted Cumulative Gain)	Whether relevant results appear near the top, with higher credit for better ranking.
MAP	Mean Average Precision, useful when there are multiple relevant documents.

Key question

Did the retriever find and rank the right evidence?

Simple Retriever Metric Examples

Assume the correct chunk appears at rank 3.

Metric	Value
Hit@1	0, because the correct chunk is not first.
Hit@3	1, because the correct chunk is in the top 3.
Hit@10	1, because the correct chunk is in the top 10.
MRR	$\frac{1}{3}$, because the first correct result is at rank 3.

Interpretation

Retriever metrics focus on the ranked list of retrieved chunks before the LLM generates an answer.

Generator Evaluation Metrics

Generator metrics evaluate the final answer produced from the retrieved context.

Metric	What it checks
Faithfulness	Is the answer supported by the retrieved context?
Answer correctness	Does the answer match the expected answer?
Answer relevance	Does the answer directly answer the question?
Completeness	Does the answer cover all required information?
Hallucination rate	How often the answer includes unsupported claims.
Citation accuracy	Whether the cited source supports the generated claim.
Groundedness	Whether the answer can be traced back to retrieved evidence.

Key question

Did the LLM use the retrieved evidence correctly?

Retriever vs Generator Evaluation

Part	Question	Example metrics
Retriever	Did we find the right evidence?	MRR, Hit@K, Recall@K, Precision@K, nDCG@K, MAP
Generator	Did we generate a good answer?	Faithfulness, correctness, relevance, hallucination rate, groundedness
Full RAG system	Did the system answer correctly using evidence?	Human review, LLM-as-judge, task success

Main lesson

RAG evaluation separates retrieval quality from answer quality.

Libraries for RAG Evaluation

Several libraries can help evaluate RAG pipelines.

Library	Used for
RAGAS	RAG-specific metrics such as faithfulness, answer relevancy, context precision, and context recall.
LangSmith	Dataset-based evaluation, tracing, experiments, human review, code-based evaluators, and LLM-as-judge evaluation.
DeepEval	Unit testing for LLM apps, with metrics such as faithfulness, answer relevancy, hallucination, and correctness.
TruLens	RAG evaluation and tracing using the RAG triad: context relevance, groundedness, and answer relevance.
BEIR	Benchmarking information retrieval models across retrieval datasets.
ranx	Ranking evaluation library for metrics such as MRR, MAP, Precision@K, Recall@K, and nDCG@K.
PyTerrier	Information retrieval experimentation and evaluation toolkit.
Human evaluation	Manual grading for correctness, completeness, clarity, and citation quality.

Congratulations!

Now you know how to use LangChain to build a RAG pipeline!

